

MACHINE LEARNING: CONCEPTS AND APPLICATIONS IN PYTHON (Part 2)

DR. SAMRAT MONDAL

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY PATNA

CROSS VALIDATION

- It is a **resampling** technique used in machine learning to assess a model's performance and generalization ability
- It involves dividing the available data into multiple **folds** or **subsets**
- Using one of these folds as a **validation set** and training the model on the remaining folds
- This process is **repeated multiple times** each time using a different fold as the validation set.
- Finally the results from each validation step are **averaged** to produce a more robust estimate of the model's performance.

HOLD OUT METHOD

- The hold-out cross-validation strategy is a simple method for evaluating a machine learning model by **splitting** the dataset into two separate sets:
 - **Training Set** – Used to train the model.
 - **Test Set** – Used to evaluate the model's performance.
- Typically, the dataset is split into **70-80%** for **training** and **20-30%** for **testing**.
- The model is **trained** on the training set and **evaluated** on the test set.
- The **test set** serves as a proxy for unseen data to estimate the model's generalization ability.

K-FOLD CROSS VALIDATION

- The dataset is randomly divided into **K subsets (or folds)**.
- The model is **trained** on K-1 folds and **tested** on the remaining 1 fold.
- This process **repeats K times**, each time using a different fold as the test set.
- The final performance is calculated by **averaging the results** from all K iterations.

STRATIFIED CROSS VALIDATION

- The dataset is divided into K folds, ensuring that each fold has a **similar class distribution** as the full dataset.
- The model is **trained** on $K-1$ folds and **tested** on the remaining fold.
- This process **repeats K times**, each time using a **different fold** for validation.
- The final performance is calculated by **averaging** the results from all K iterations.

SCENARIO

Suppose we are conducting a survey in a school with 1000 students, where 60% are male and 40% are female. If we want to select a sample of 100 students while maintaining the same gender ratio, we use stratified sampling.

Divide the population into strata (groups):

Male students(60%): 600 students

Female students(40%): 400 students

Determine the sample size for each group :

60% of 100 = 60 males

40% of 100 = 40 females

Randomly select students from each group:

Randomly pick 60 males from the 600 male students.

Randomly pick 40 females from the 400 female students.

DECISION TREE

- Decision trees can be applied to both **regression** and **classification** problems
- **Regression Tree:**
 - Used for **continuous** outputs (e.g., Predicting house prices).
 - Splitting Criteria: Mean Squared Error (MSE), Mean Absolute Error (MAE).
- **Classification Tree:**
 - Used for **categorical** outputs (e.g., Yes/No, Spam/Not Spam).
 - Splitting Criteria: Gini Impurity, Entropy (Information Gain).

Decision Tree

Mix of discrete and continuous data

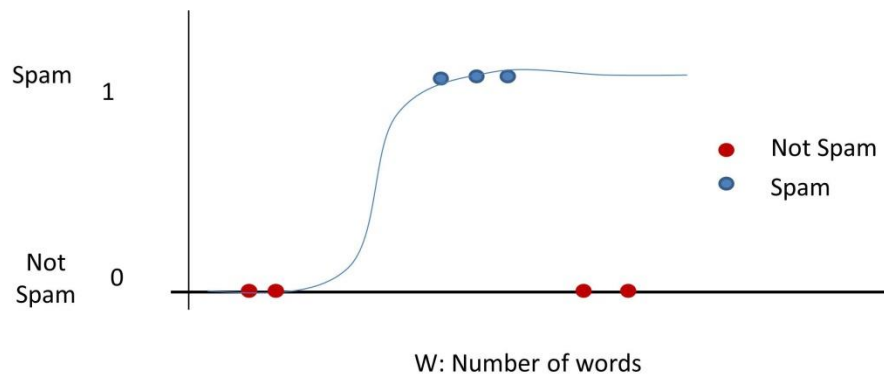
Based on three features-
Any links present, 'Free'
Keyword present and
Number of Words, predict
whether the email is
spam or not



Any links present	'Free' Keyword present	Number of Words	Spam Email
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

What about Logistic regression?

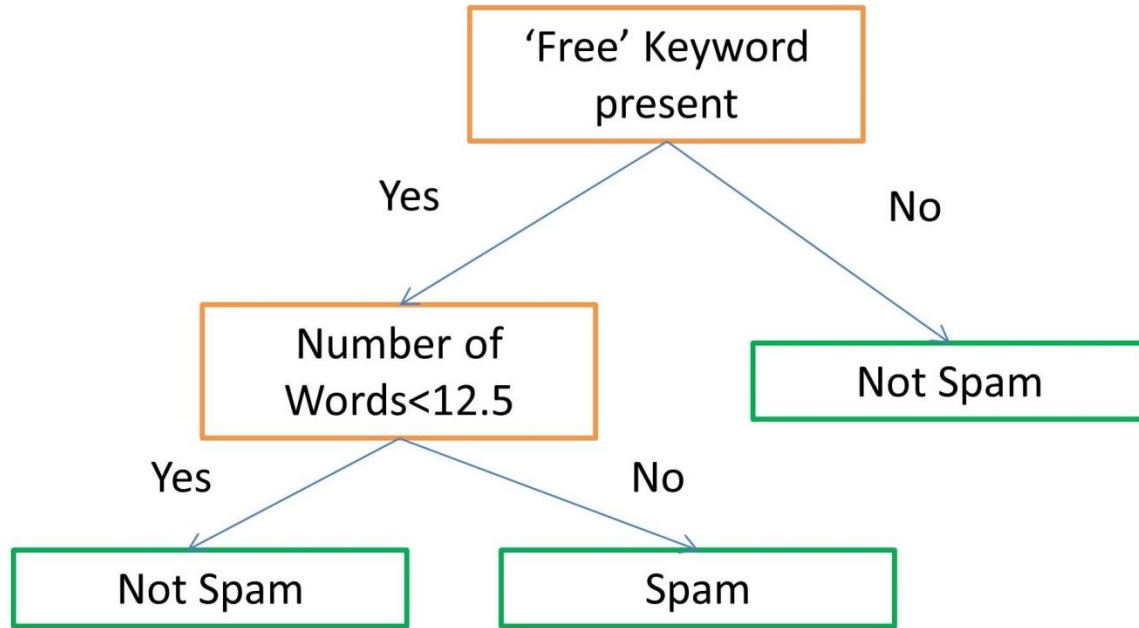
Number of Words	Spam Email
7	No
12	No
18	Yes
35	Yes
38	Yes
50	No
83	No



Logistic regression would not do well

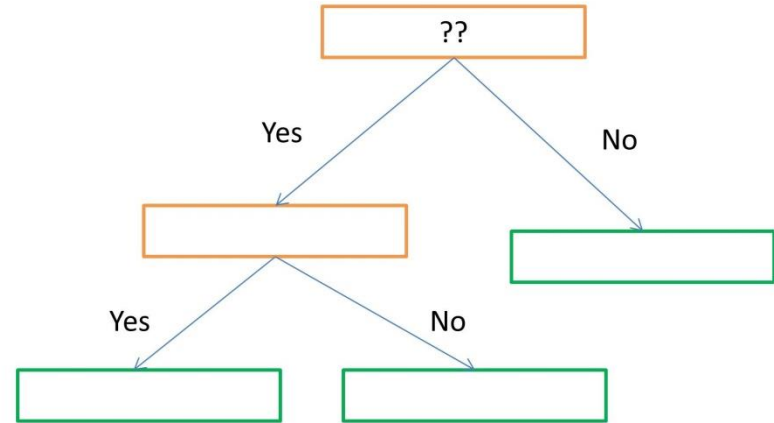
Let's see how decision tree handles it

Decision tree for Classification



How to decide the attribute at each node?

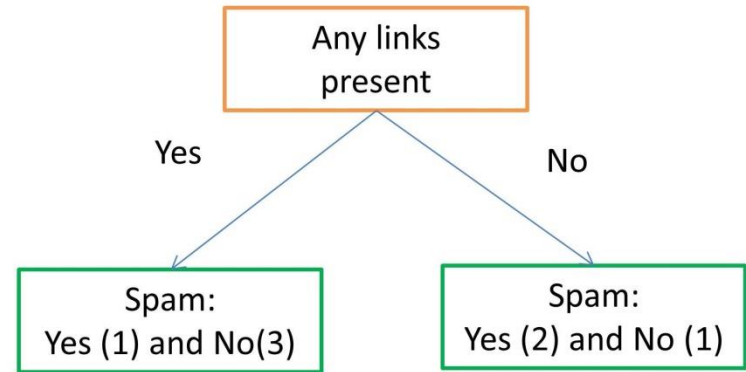
Any links present	'Free' Keyword present	Number of Words	Spam Email
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



Any links

How well **Any Links Present** feature predicts whether an email is spam or not

Any links present	'Free' Keyword present	Number of Words	Spam Email
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

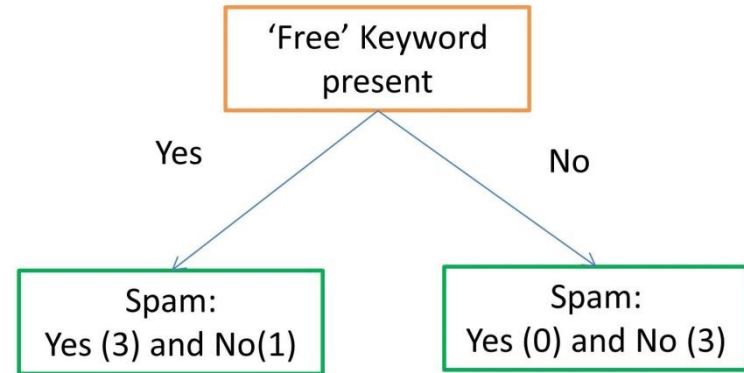


Leaves that contain mixture of classes are **impure**

Free keyword

How well **Free Keyword Present** feature predicts whether an email is spam or not

Any links present	'Free' Keyword present	Number of Words	Spam Email
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No



Leaves that contain only one class is **pure**

Measures of node impurity

- To quantify **impurity** of the leaves and tree, **Gini** impurity can be used
- Entropy is another option
- We will see Gini as the other one provides similar results

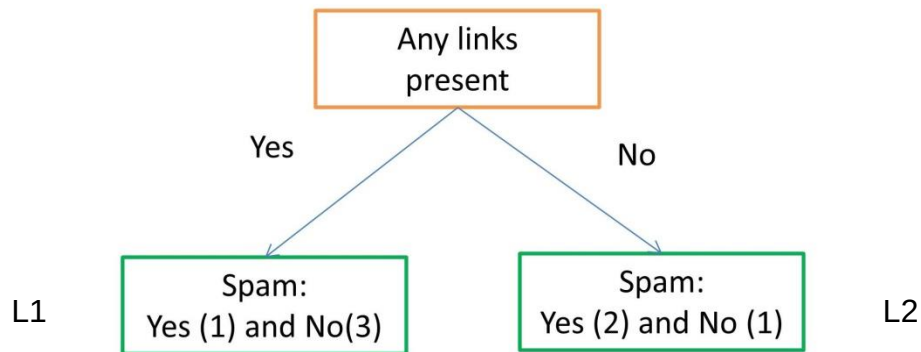
Measures of Node Impurity

$$Gini\ Index = 1 - \sum_{i=0}^{c-1} p_i(t)^2$$

Where $p_i(t)$ is the frequency of class i at node t , and c is the total number of classes

$$Entropy = - \sum_{i=0}^{c-1} p_i(t) \log_2 p_i(t)$$

Gini computation



$$Gini(L1) = 1 - (P_{yes})^2 - (P_{No})^2$$

$$Gini(L1) = 1 - \left(\frac{1}{1+3}\right)^2 - \left(\frac{3}{1+3}\right)^2$$

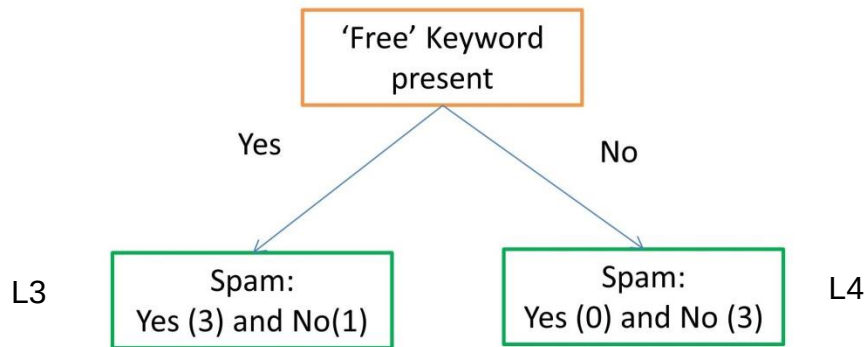
$$Gini(L1) = 0.375$$

$$Gini(L2) = 1 - (P_{yes})^2 - (P_{No})^2$$

$$Gini(L2) = 1 - \left(\frac{2}{2+1}\right)^2 - \left(\frac{1}{2+1}\right)^2$$

$$Gini(L2) = 0.4555$$

Gini computation



$$Gini(L3) = 1 - (P_{yes})^2 - (P_{No})^2$$

$$Gini(L3) = 1 - \left(\frac{3}{1+3}\right)^2 - \left(\frac{1}{1+3}\right)^2$$

$$Gini(L3) = 0.375$$

$$Gini(L4) = 1 - (P_{yes})^2 - (P_{No})^2$$

$$Gini(L4) = 1 - \left(\frac{0}{0+3}\right)^2 - \left(\frac{3}{0+3}\right)^2$$

$$Gini(L4) = 0$$

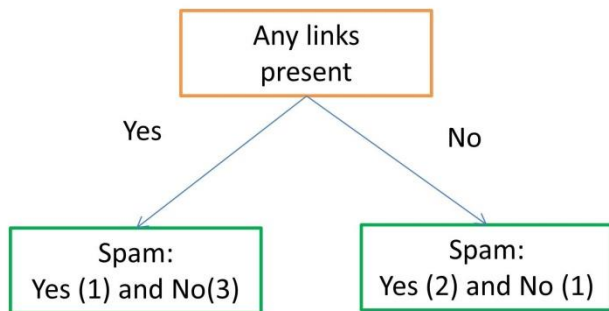
Total Gini impurity of a split

- When a node p is split into k partitions (children)

$$GINI_{split} = \sum_{i=1}^k \frac{n_i}{n} GINI(i)$$

- where,
 n_i = number of records at child i ,
 n = number of records at parent node p .

Gini Split



Split1

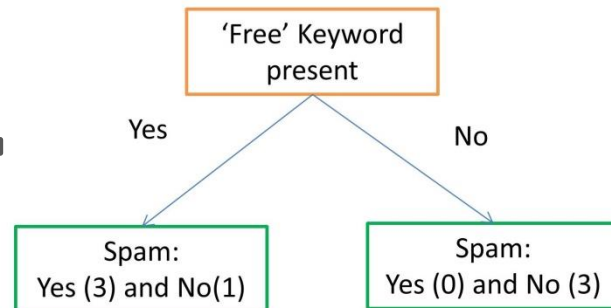
$$Gini(Split1) = \left(\frac{4}{4+3}\right) 0.375 + \left(\frac{3}{4+3}\right) 0.444$$

$$Gini(Split1) = 0.405$$

$$Gini(Split2) = \left(\frac{4}{4+3}\right) 0.375 + \left(\frac{3}{4+3}\right) 0$$

$$Gini(Split2) = 0.214$$

Split2



Lower Gini impurity for a split is preferred

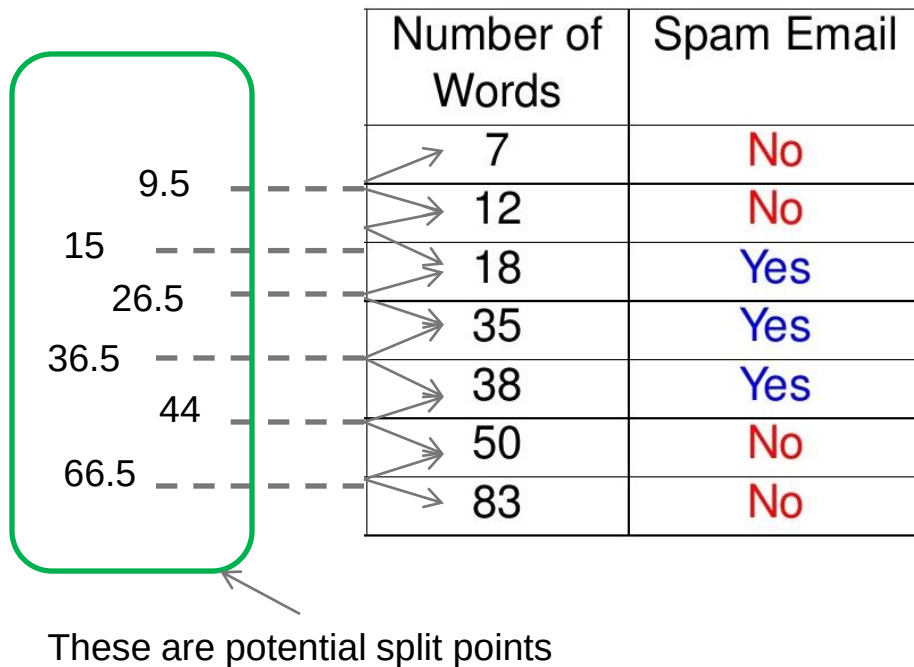
Gini for continuous attribute

Any links present	'Free' Keyword present	Number of Words	Spam Email
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

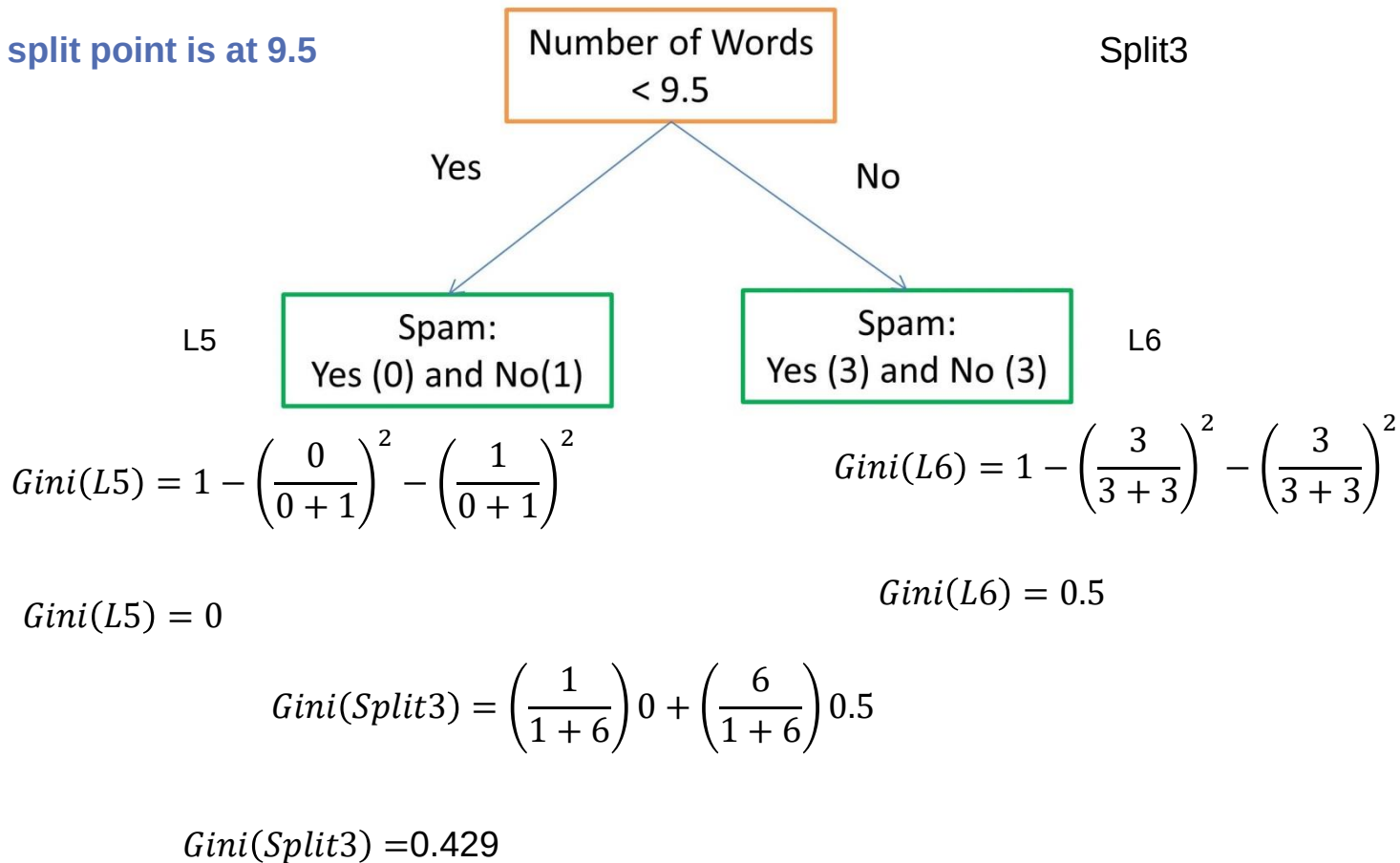
Sort the attribute values first

In this case the values of **Number of Words** is already sorted

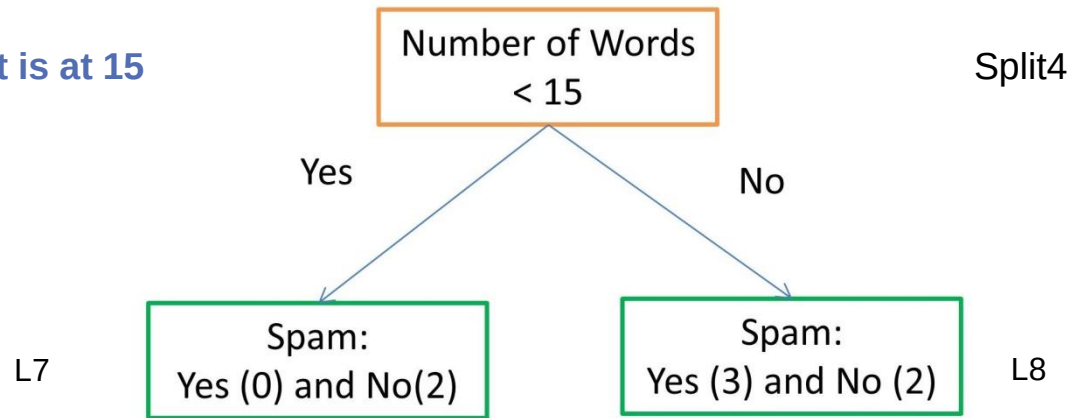
Calculate the average of each adjacent pairs



When split point is at 9.5



When split point is at 15



$$Gini(L7) = 1 - \left(\frac{0}{0+2}\right)^2 - \left(\frac{2}{0+2}\right)^2$$

$$Gini(L7) = 0$$

$$Gini(L8) = 1 - \left(\frac{3}{3+2}\right)^2 - \left(\frac{2}{3+2}\right)^2$$

$$Gini(L8) = 0.48$$

$$Gini(Split4) = \left(\frac{2}{2+5}\right) 0 + \left(\frac{5}{2+5}\right) 0.48$$

$$Gini(Split4) = 0.343$$

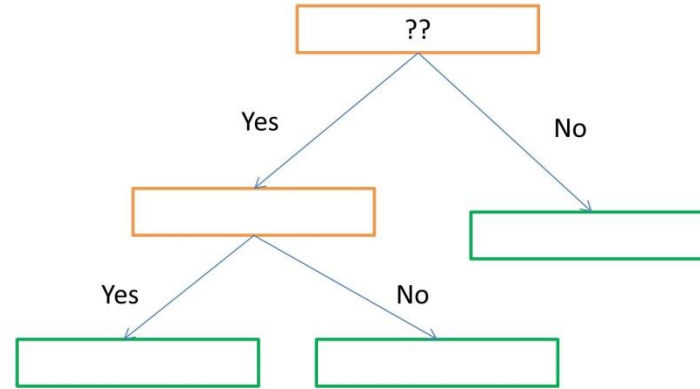
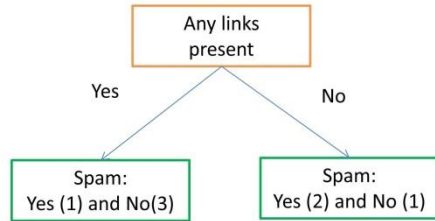
Gini for continuous attribute

Gini Split			Number of Words	Spam Email
			7	No
0.429	←	9.5	12	No
0.343	←	15	18	Yes
0.476	←	26.5	35	Yes
0.476	←	36.5	38	Yes
0.343	←	44	50	No
0.429	←	66.5	83	No

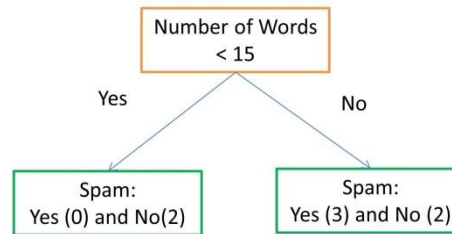
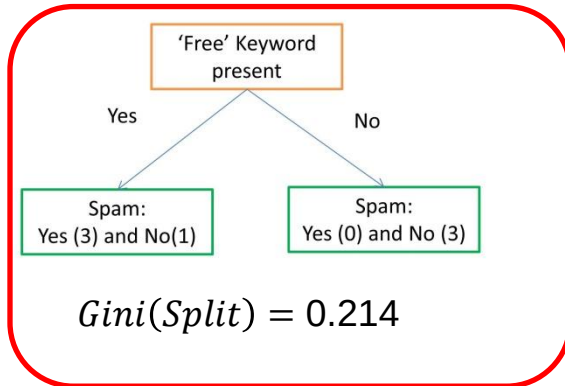
Split point with lowest Gini impurity can be picked

In this case both 15 and 44 give lowest Gini impurity. Anyone can be picked

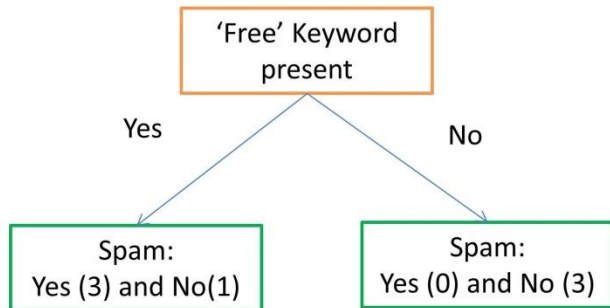
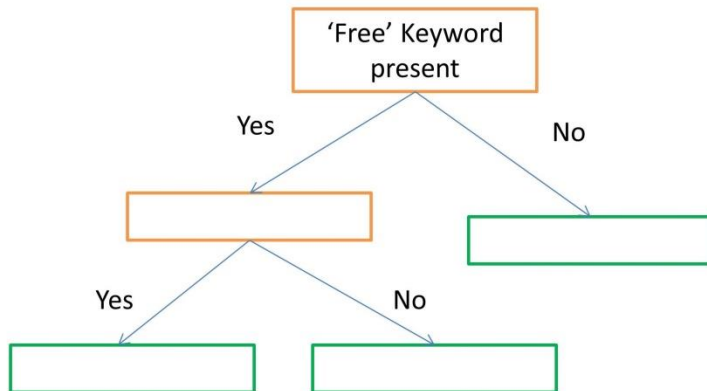
How to decide the attribute at each node?



$$Gini(Split) = 0.405$$



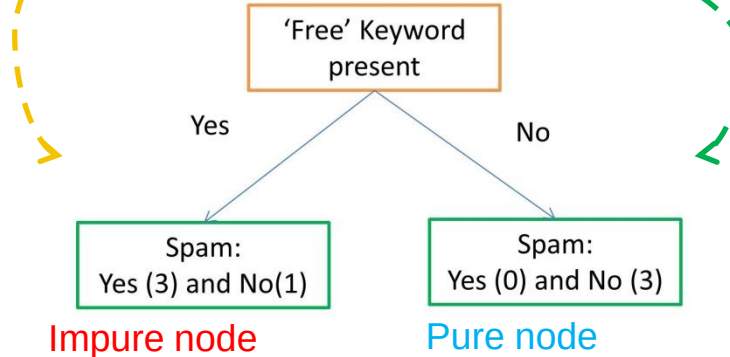
$$Gini(Split) = 0.343$$



Any links present	'Free' Keyword present	Number of Words	Spam Email
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

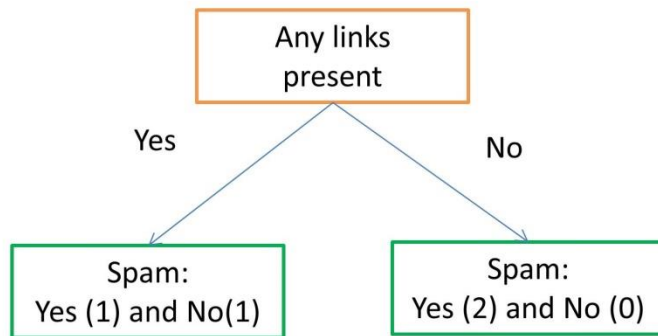
Any links present	'Free' Keyword present	Number of Words	Spam Email
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

Any links present	'Free' Keyword present	Number of Words	Spam Email
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

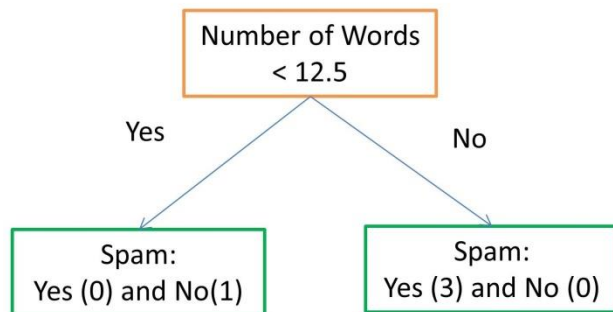


Now, split based on other features

Becomes a leaf node



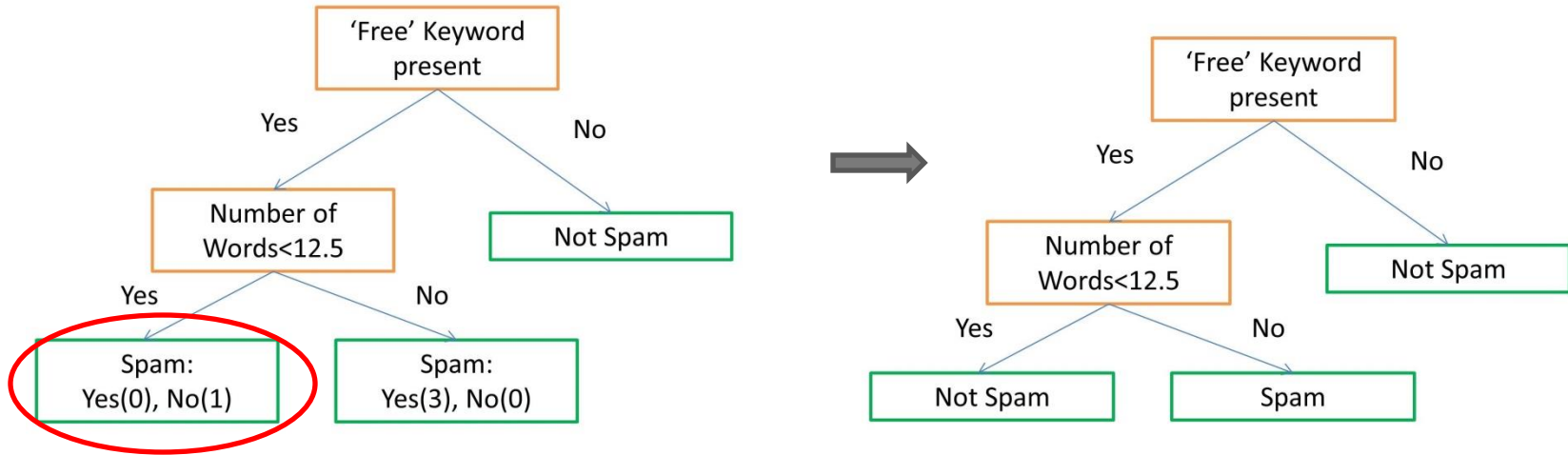
$$Gini(Split) = 0.25$$



$$Gini(Split) = 0$$

Any links present	'Free' Keyword present	Number of Words	Spam Email
Yes	Yes	7	No
Yes	No	12.5	No
No	Yes	18	Yes
No	Yes	26.5	Yes
Yes	Yes	36.5	Yes
Yes	No	50	No
No	No	83	No

Final decision tree



As only few entry of training data make this as leaf, the tree may not do well for future data

- Tree pruning is used
- Putting on the minimum limits on the number entries per leaf.
 - E.g. 3 per leaf node
- However, it is difficult to know this number in advance.
- This needs to be determined based on cross validation
- Majority vote can be used for the impure nodes

Decision Tree

Advantages:

- Relatively inexpensive to construct
- Extremely fast at classifying unknown records
- Easy to interpret for small-sized trees
- Robust to noise (especially when methods to avoid overfitting are employed)
- Can easily handle redundant attributes
- Can easily handle irrelevant attributes (unless the attributes are **interacting**)

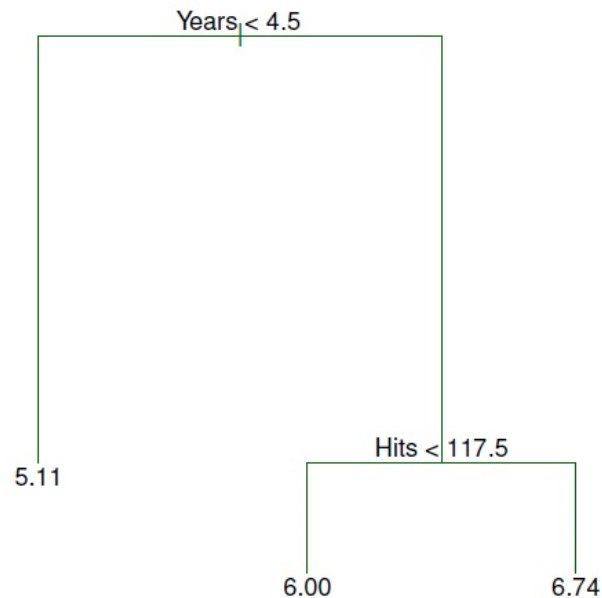
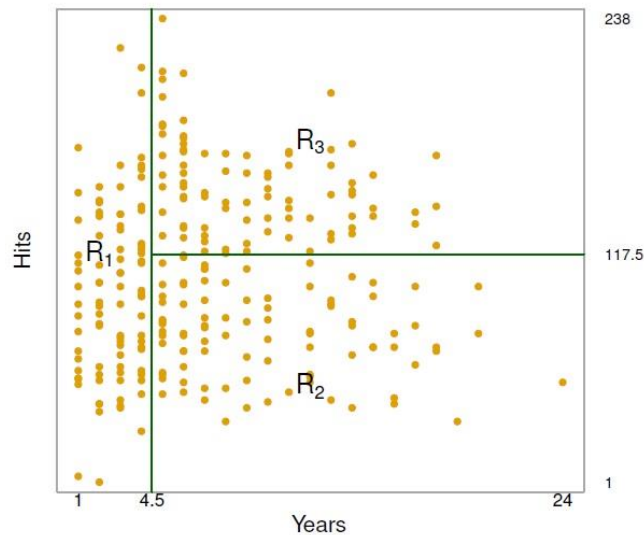
Disadvantages: .

- Due to the greedy nature of splitting criterion, **interacting** attributes (that can distinguish between classes together but not individually) may be passed over in favor of other attributed that are less discriminating.
- Each decision boundary involves only a single attribute

REGRESSION TREE

- Let's consider **Hitters** data
 - Contains baseball player statistics from Major League Baseball (MLB) and is often used for predicting salaries
- **Dataset Size:** 322 players
- **Objective:** Predict a player's salary based on their past performance
- **Target Variable:** Salary (continuous variable, in thousands of dollars)

REGRESSION TREE FOR HITTERS

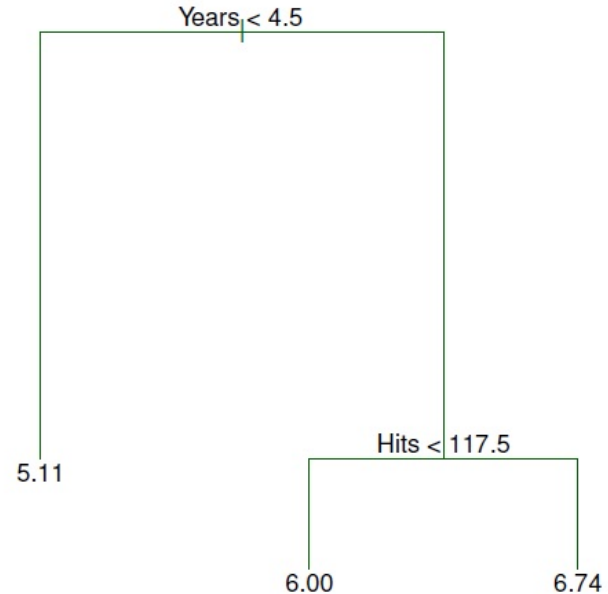
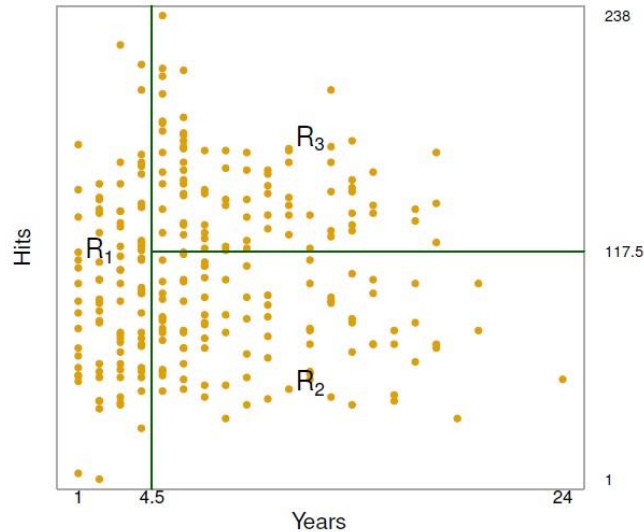


$$R_1 = \{X | Years < 4.5\}$$

$$R_2 = \{X | Years \geq 4.5 \text{ AND } Hits < 117.5\}$$

$$R_3 = \{X | Years \geq 4.5 \text{ AND } Hits \geq 117.5\}$$

REGRESSION TREE FOR HITTERS



$$\text{Salary}(R_1) = 1000 * e^{5.11} = \$165174$$

$$\text{Salary}(R_2) = 1000 * e^{6.00} = \$402834$$

$$\text{Salary}(R_3) = 1000 * e^{6.74} = \$845346$$

PROCESS OF BUILDING THE REGRESSION TREE

- Divide the predictor space, i.e., the set of possible values for $X_1, X_2, \dots, X_p$ — into J distinct and non-overlapping regions, $R_1, R_2, \dots, R_J$
- For every observation that falls into the region R_j , we make the same prediction, which is simply the mean of the response values for the training observations in R_j
- In theory, the regions could have any shape. However, for simplicity and ease of interpretation, we choose to divide the predictor space into high-dimensional **rectangles**, or **boxes**
- The goal is to find boxes $R_1, \dots, R_J$ that minimize the RSS, given by

$$\sum_{j=1}^J \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2$$

Mean response of the training observations within jth box

TOP-DOWN, GREEDY APPROACH

- Unfortunately, it is computationally infeasible to consider every possible partition of the feature space into J boxes
- For this reason, we take a **top-down**, **greedy** approach that is known as recursive binary splitting.
 - **Top-down**: because it begins at the **top of the tree** (at which point all observations belong to a single region) and then successively splits the predictor space; each split is indicated via two new branches further down on the tree.
 - **Greedy**: at each step of the tree-building process, the **best** split is made **at that particular step**, rather than looking ahead and picking a split that will lead to a better tree in some future step.

RECURSIVE BINARY SPLITTING

- Recursive binary splitting, first select the predictor X_j and the cutpoint s such that splitting the predictor space into the regions $R_1(j,s)$ and $R_2(j,s)$ leads to greatest possible reduction in RSS

$$R_1(j, s) = \{X | X_j < s\} \quad R_2(j, s) = \{X | X_j \geq s\}$$

- We optimize the value of j and s such that following is minimized

$$\sum_{i: x_i \in R_1(j, s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i: x_i \in R_2(j, s)} (y_i - \hat{y}_{R_2})^2$$

Mean response of the training
observations in $R_1(j, s)$

Mean response of the training
observations in $R_2(j, s)$

REGRESSION TREE: SPLITTING CRITERION

- Regression trees use **variance reduction** techniques to determine the best split
- The split with the highest MSE reduction is selected
- Mean Squared Error (MSE) Reduction**

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

actual mean

$$\Delta MSE = MSE_p - (w_L MSE_L + w_R MSE_R)$$

Reduction in MSE MSE in Parent MSE in left child MSE in right child

Weight or proportion in left child Weight or proportion in right child

The diagram illustrates the components of the Mean Squared Error (MSE) reduction formula. At the top, the MSE formula is shown: $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$. Arrows point from the labels 'actual' and 'mean' to y_i and $\bar{y}$ respectively. Below this, the reduction formula is shown: $\Delta MSE = MSE_p - (w_L MSE_L + w_R MSE_R)$. Arrows point from the labels 'Reduction in MSE', 'MSE in Parent', 'MSE in left child', and 'MSE in right child' to ΔMSE , MSE_p , MSE_L , and MSE_R respectively. Finally, arrows point from the labels 'Weight or proportion in left child' and 'Weight or proportion in right child' to w_L and w_R respectively.

PARAMETERS VS HYPERPARAMETERS

- **Parameter:** Internal values learned from data during training
 - Automatically learned from the training data
 - E.g. in the linear regression **intercept**, **slope** coefficient
- **Hyperparameter:** External settings set before training to control learning
 - Manually set by the user
 - E.g. Some hyperparameters for decision tree are **max_depth**, **min_samples_split**, **min_samples_leaf**

Ensemble Method

- An *ensemble* method is an approach that combines many simple “building block” models in order to obtain a single and potentially very powerful ensemble model.
- These simple building block models are sometimes known as *weak learners*, since they may lead to mediocre predictions on their own.

Ensemble Learning

- **Bagging:** Learners are trained *independently* on random subsets. (Parallel)
- **Boosting:** Learners are trained *sequentially* — each focuses on correcting previous mistakes.

AdaBoost

- Train a weak model (e.g., a shallow decision tree).
- Evaluate its errors.
- Give more weight to misclassified samples.
- Train the next model, focusing on the hard examples.
- Combine all models with weighted voting (classification) or averaging (regression).

Gradient Boosting

- Start with a simple model (e.g., a single leaf predicting the mean value).
- Calculate residuals (actual – predicted).
- Train a new tree on these residuals.
- Add the new tree's predictions (scaled by learning rate) to the old predictions.
- Repeat until residuals are small or max trees reached.

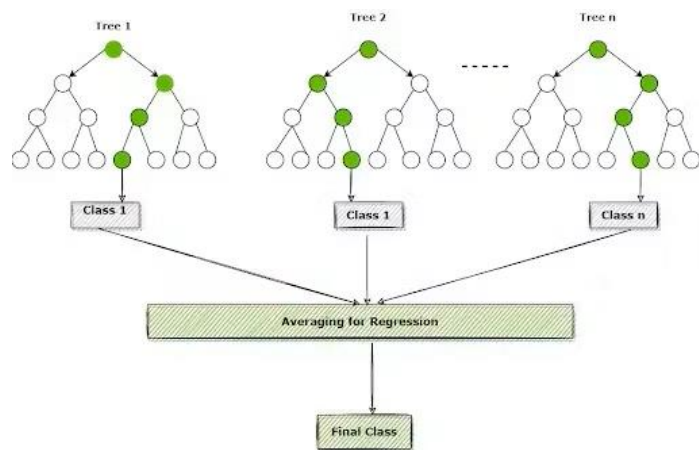
XGBoost

- XGBoost still builds trees sequentially like Gradient Boosting.
- But it adds:
 - Regularization (L1 & L2 penalties) → smoother models
 - Column sampling → faster and less correlated trees
 - Advanced tree pruning → avoids overfitting
 - Optimized loss functions → supports logistic, regression, etc.

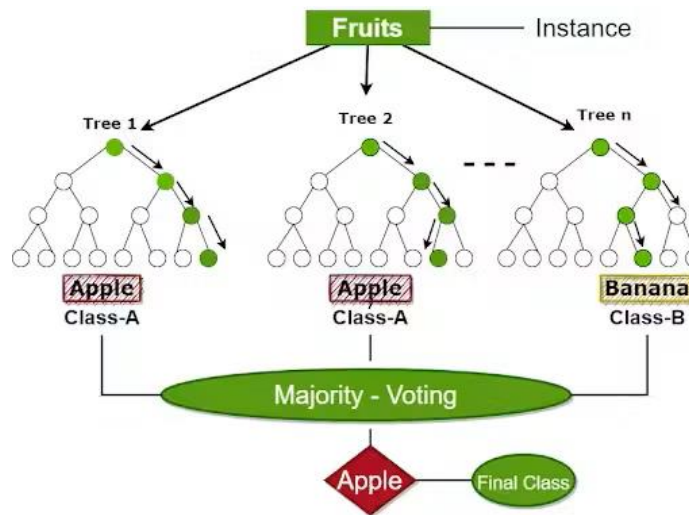
Random Forest

- A Random Forest is a collection of decision trees that work together to make predictions
- It builds multiple decision trees using random samples of the data.
- Each tree is trained on a different subset of the data.

Random Forest



For regression



For classification

Advantages of random forest

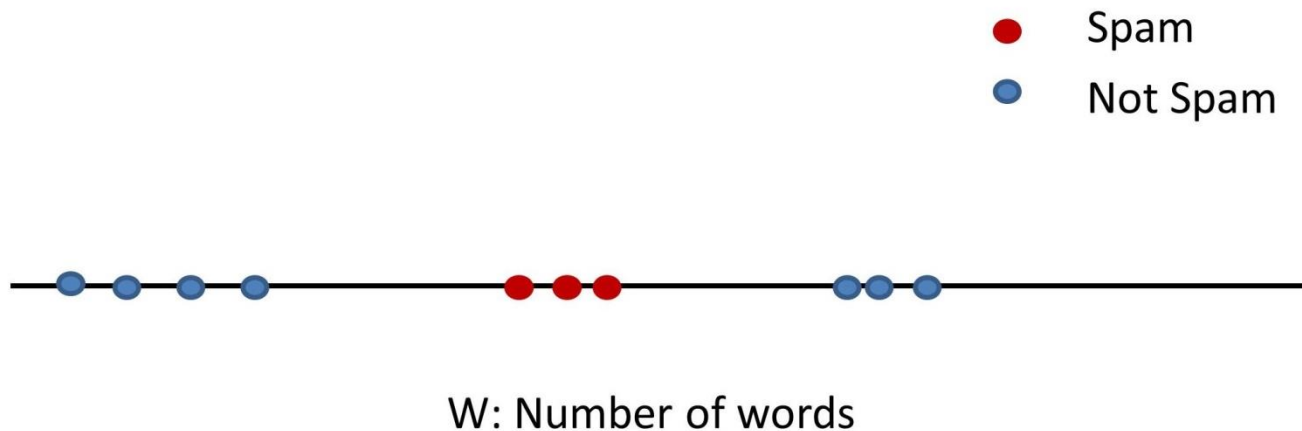
- Random Forest Algorithm eliminates **overfitting** as the result is based on a majority vote or average.
- Each decision tree formed is independent of the others, demonstrating the **parallelization** property
- Because the average answers from a vast number of trees are used, it is **highly stable**
- It preserves **diversity** by not considering all features while creating each decision tree, however, this is not true in all circumstances
- It is not affected by the **dimensionality curse**. The feature space is minimized because each tree does not consider all properties.

Random Forests: points to consider

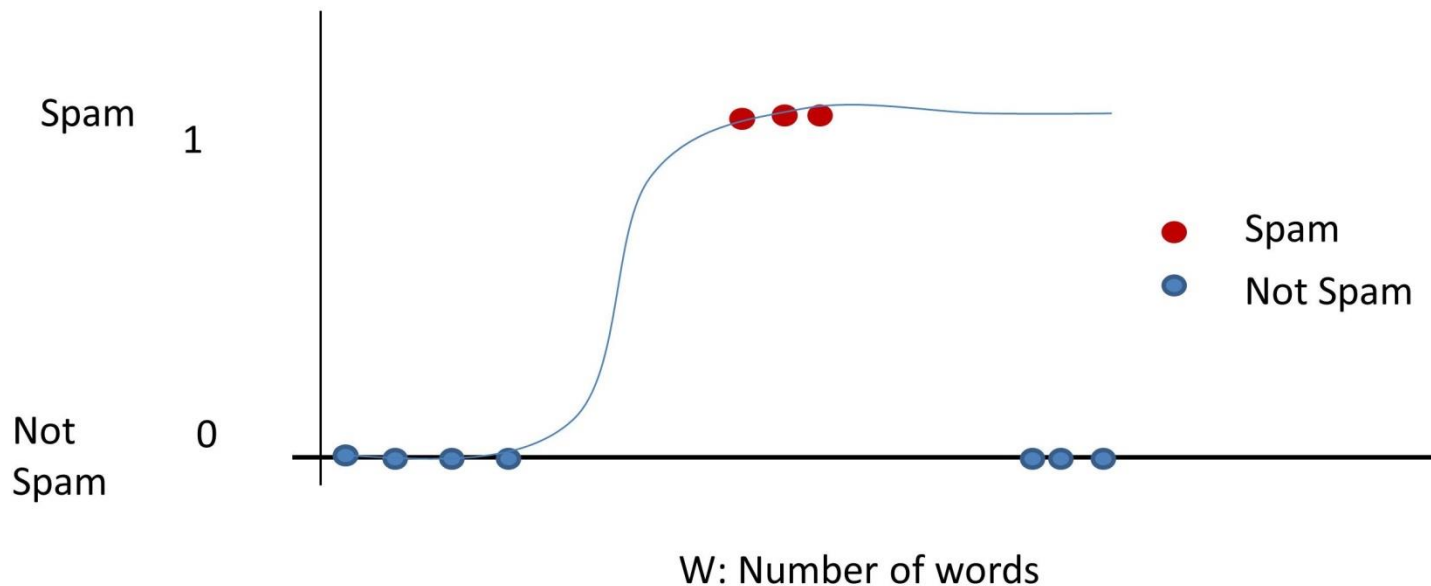
- When compared to decision trees, where decisions are determined by following the tree's route,
 - the random forest is much more complex
- Due to its complexities, training time is longer than for other models.
 - Each decision tree must generate output for the supplied input data whenever it needs to make a prediction.
- Key Hyperparameters:
 - Number of trees, maximum depth of the tree, minimum samples needed to split a node, number of features considered per split

Support Vector Machine

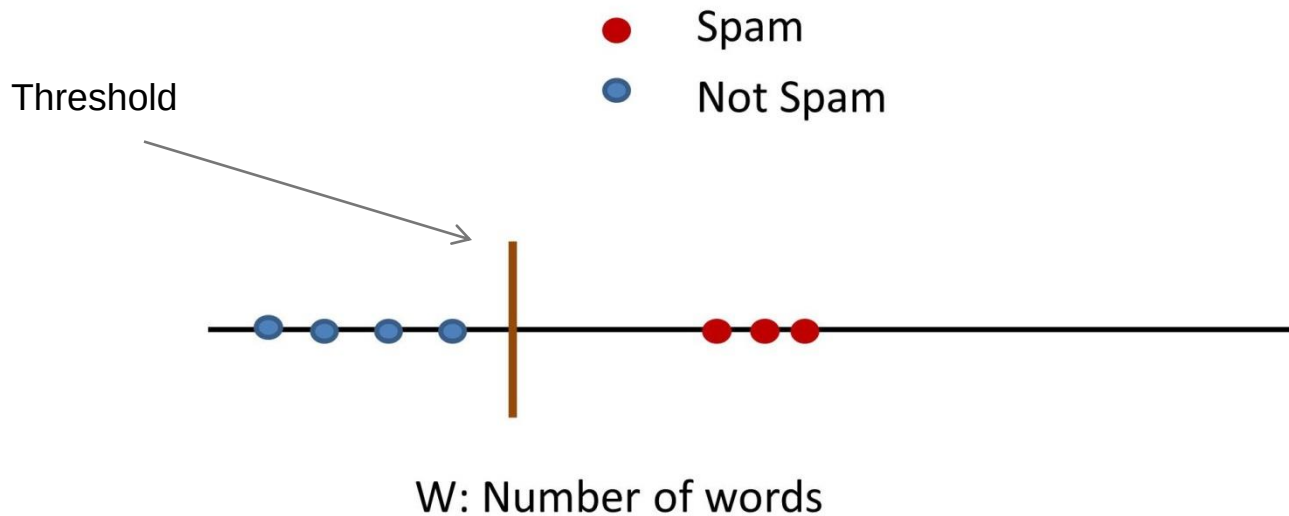
Support Vector Machine



Fitting one S-shaped curve



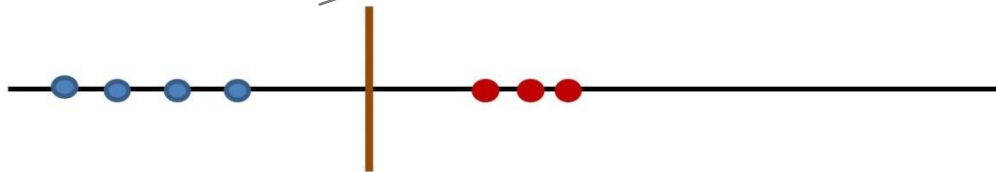
Support Vector Classifier



SVM

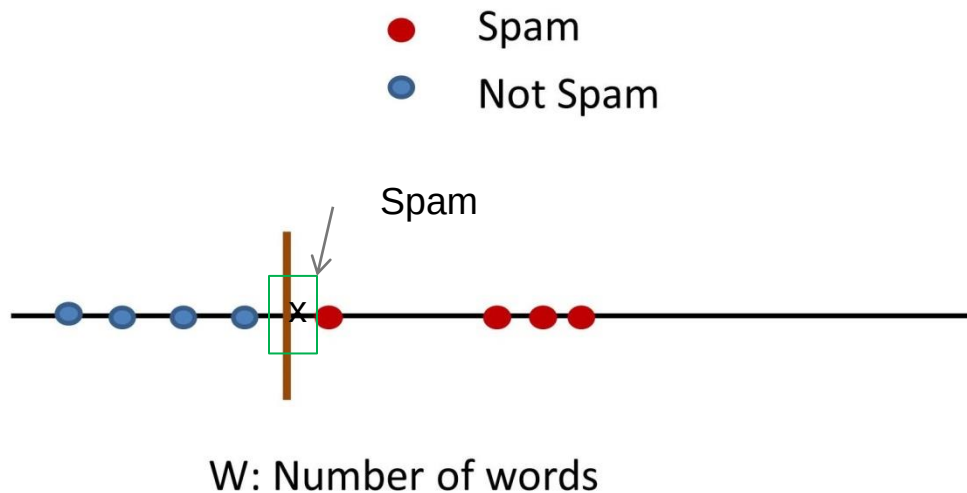
Threshold is the
midpoint

● Spam
● Not Spam

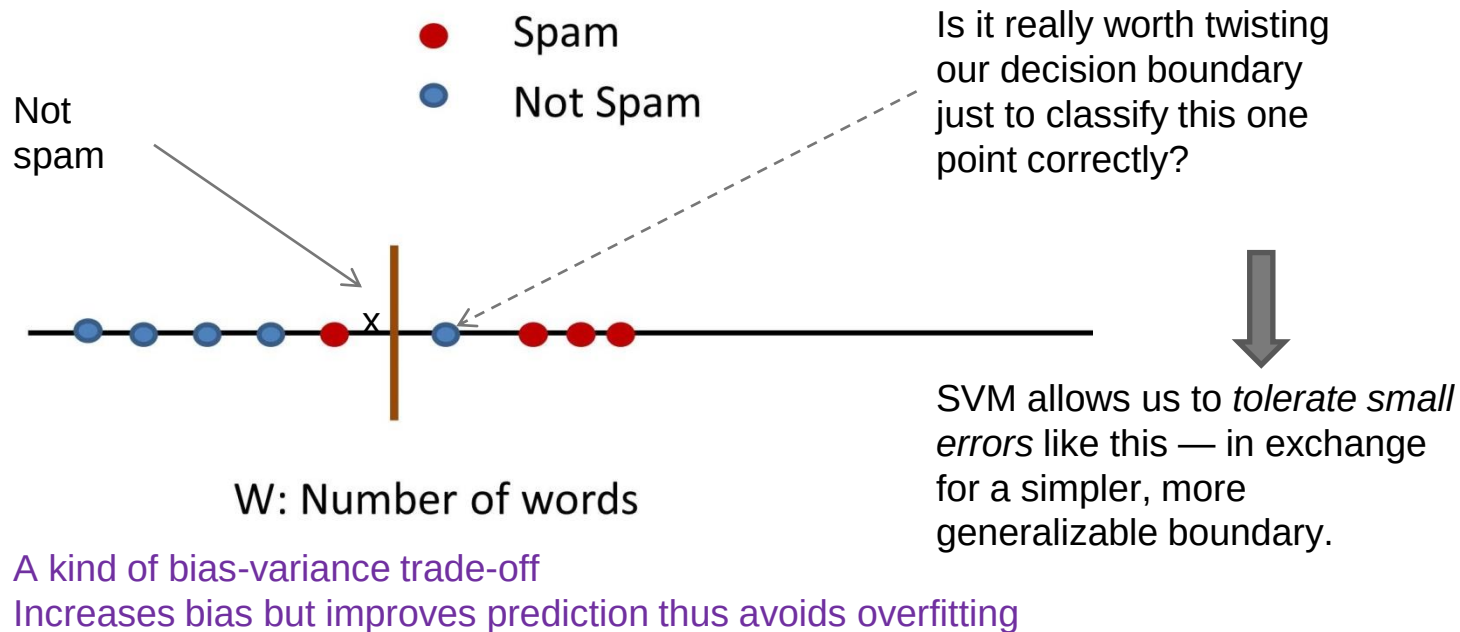


W: Number of words

With outlier

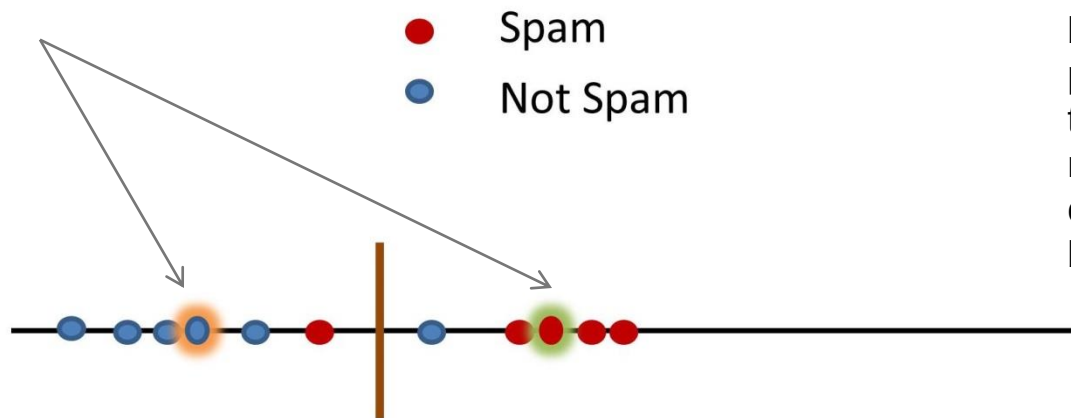


Allow some misclassification



Support Vector Classifier

Threshold can be put halfway between these pair of points

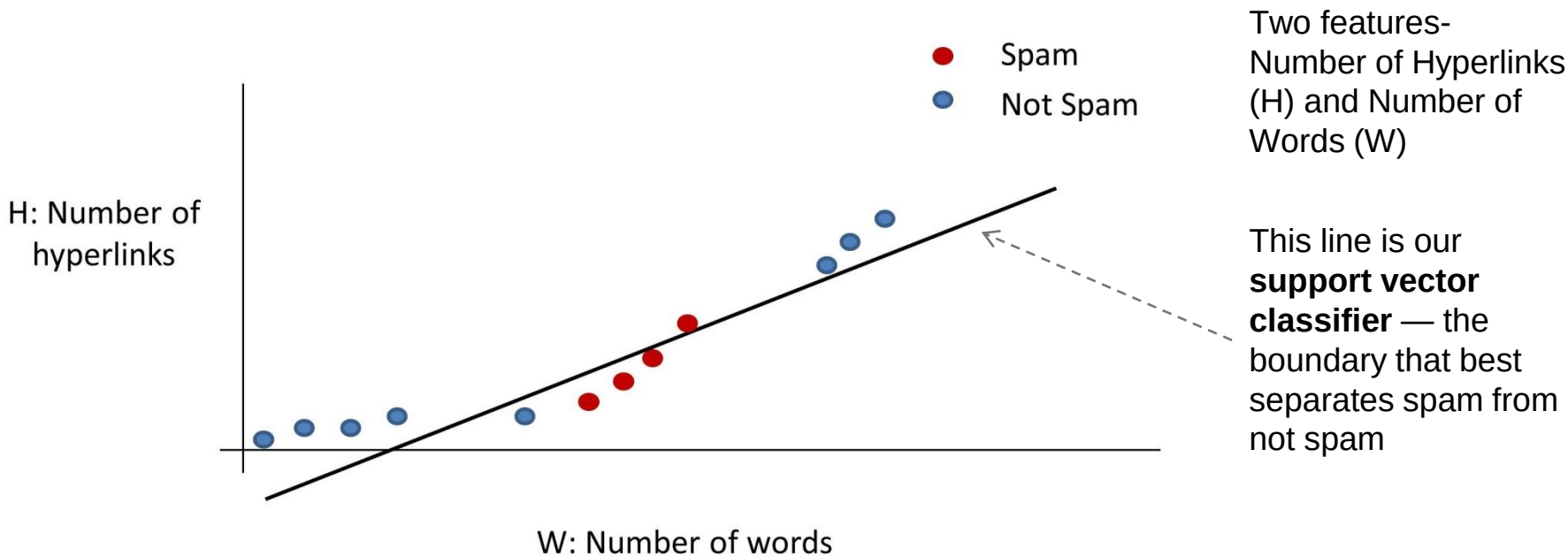


Cross validation could be used to decide the pair of points to define the threshold and how many misclassification of the training data can be allowed

W: Number of words

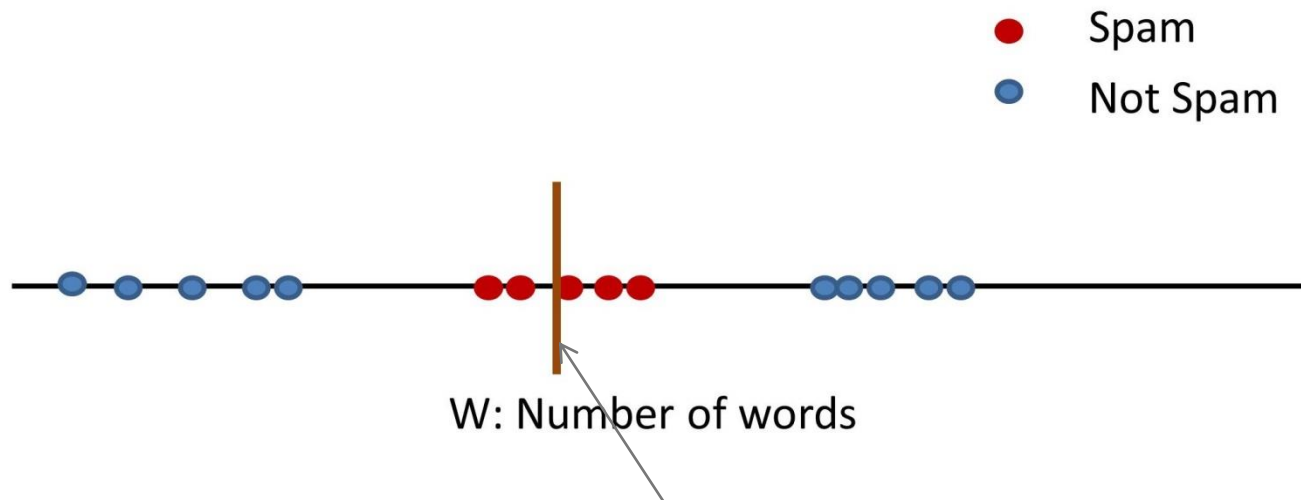
In this case, support vector classifier is just a point

Two dimensional data



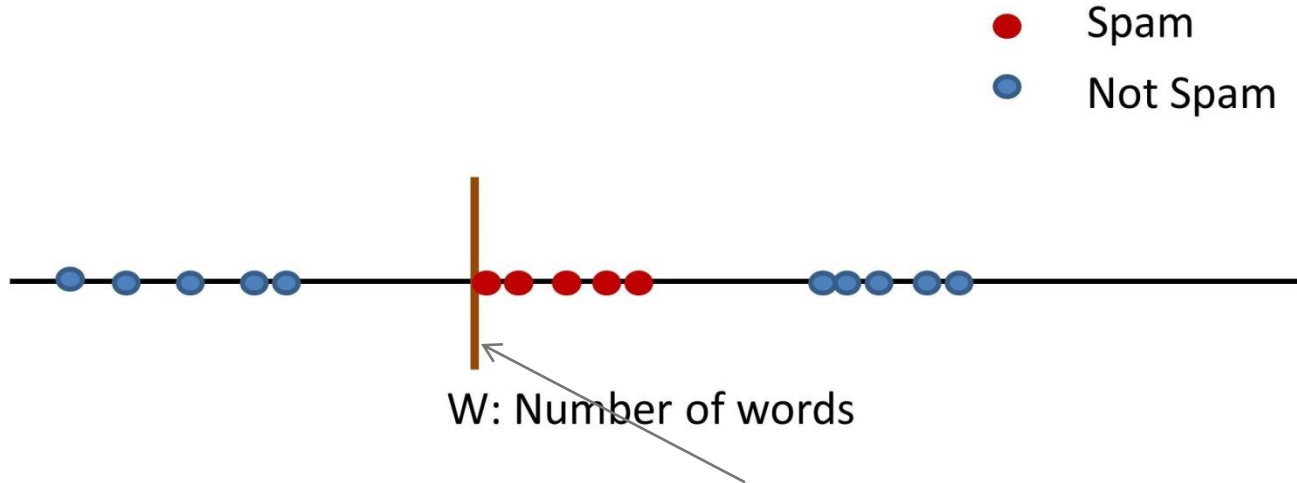
Support vector classifier is a straight line

Support Vector Classifier: issues



Support vector classifier will cause a lot of misclassifications

Support Vector Classifier: issues

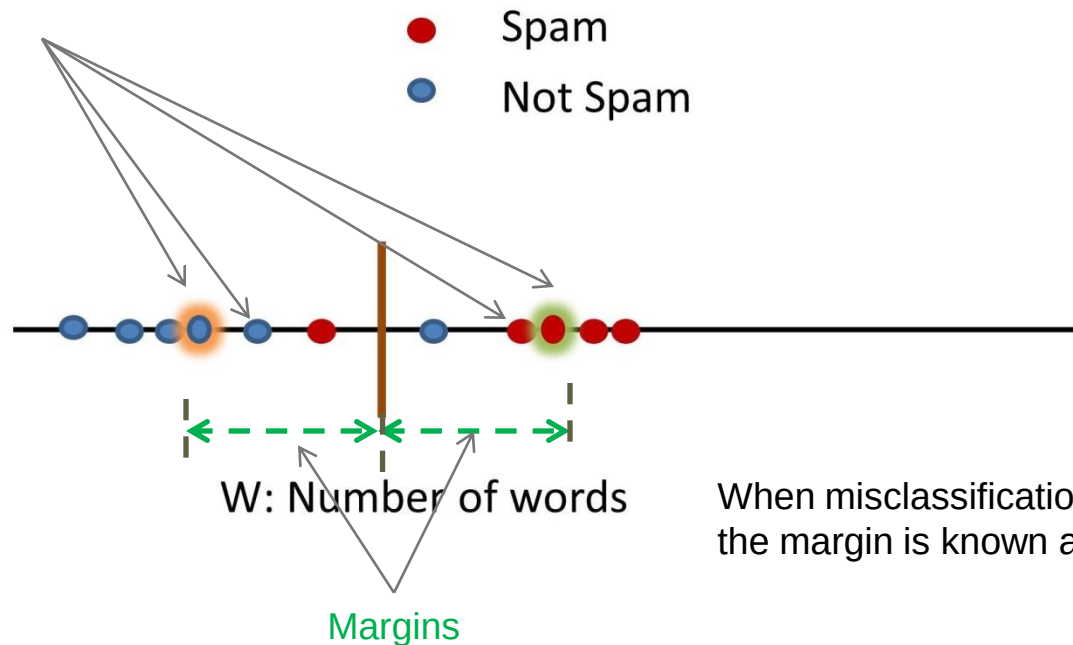


Even if the support vector classifier is shifted there will be misclassifications

Support Vector Classifier

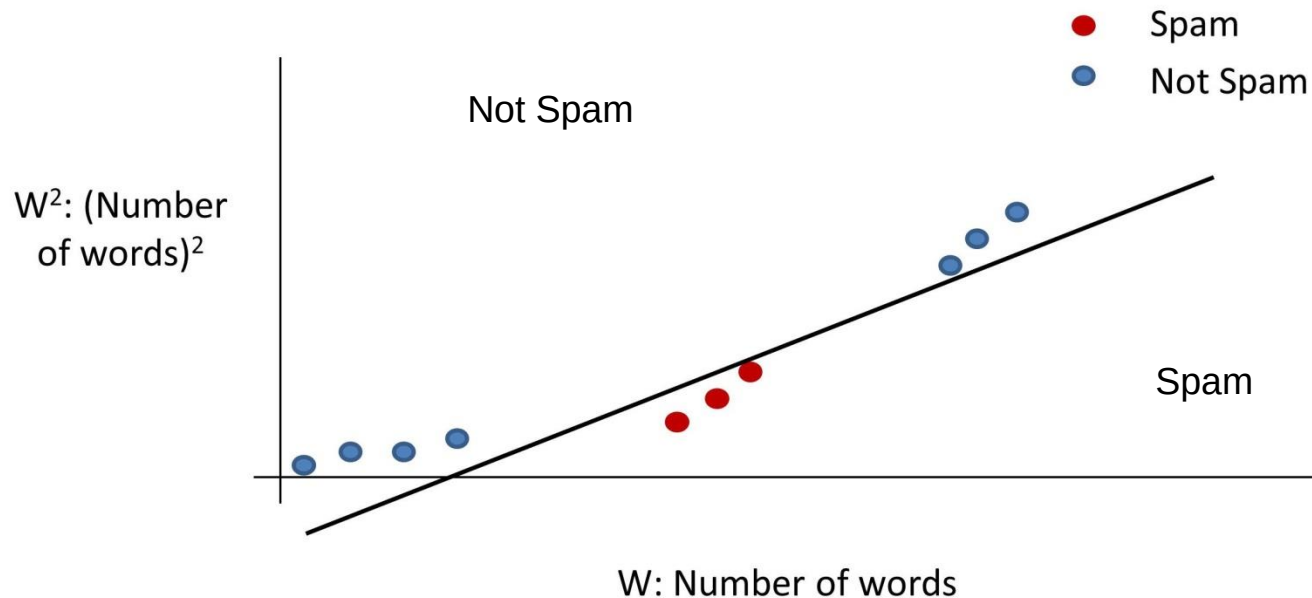
Support Vectors: Points which are closer to the threshold

Support Vectors: Points that define the thresholds



When misclassifications are allowed then the margin is known as **Soft Margin**

Support Vector Machine



What we have done

- Start with the low dimensional data
 - E.g. W : number of words
- Use existing data to create a higher dimension
 - E.g. W^2 : (number of words)²
- Find support vector classifier that separates the data into separate groups

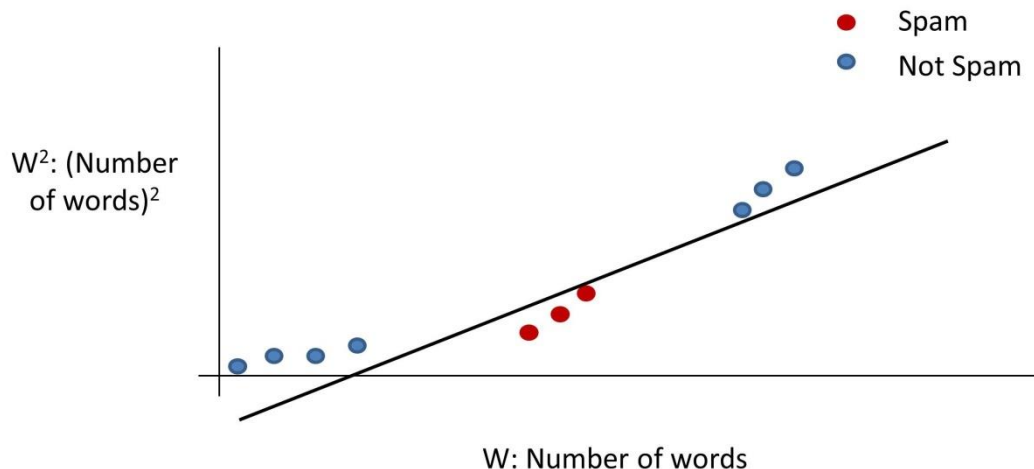
Kernel Function

- For generating higher dimensional data, we can use W^3 , W^4 , , $\pi W^{0.5}$,... instead of W^2
- How to decide which function to use for the transformation?
- SVM uses **Kernel Function**
 - which systematically find SVC in higher dimensions
- Types of Kernel Functions
 - Polynomial Kernel
 - Radial Kernel (or Radial Basis Function)

Polynomial Kernel

- Polynomial kernel: $(a * b + r)^d$

a and b; refer to two different observations of the data



r; determines the coefficient of the polynomial

d; determines the degree of the polynomial

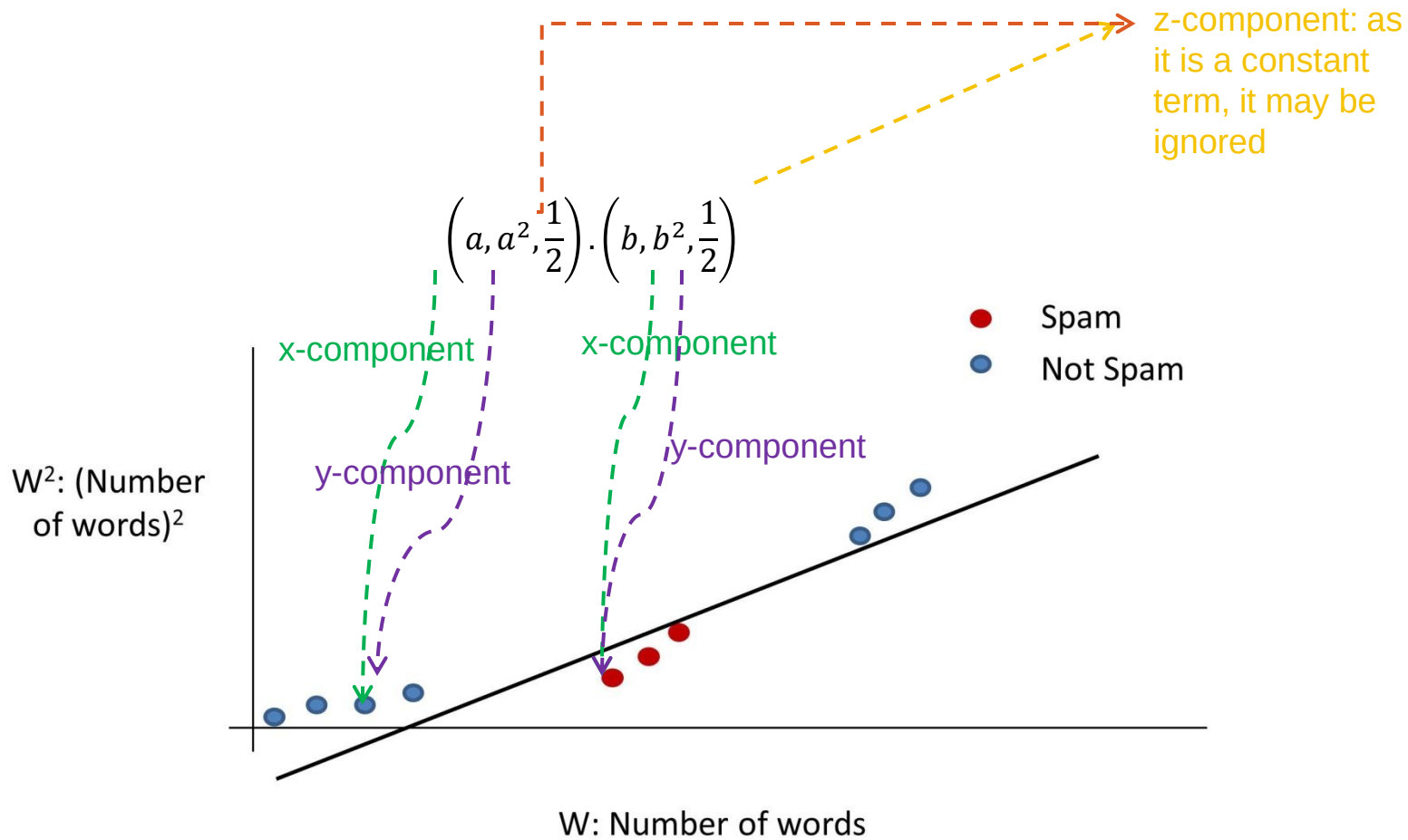
Polynomial Kernel: $(a * b + r)^d$

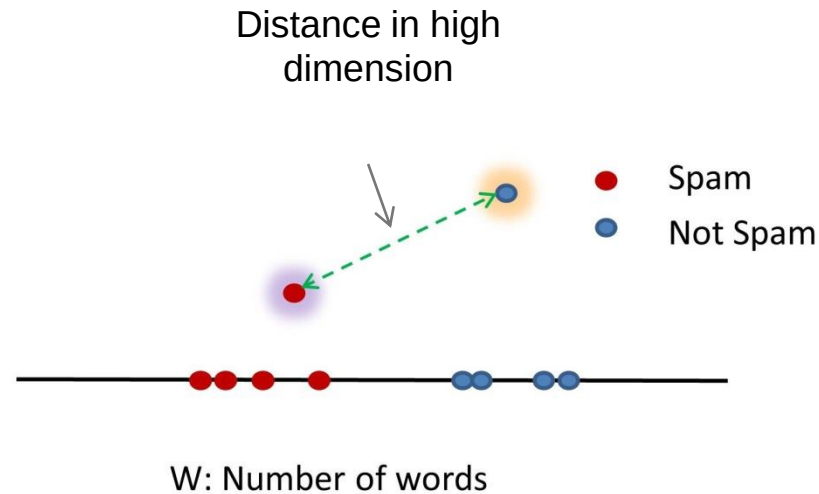
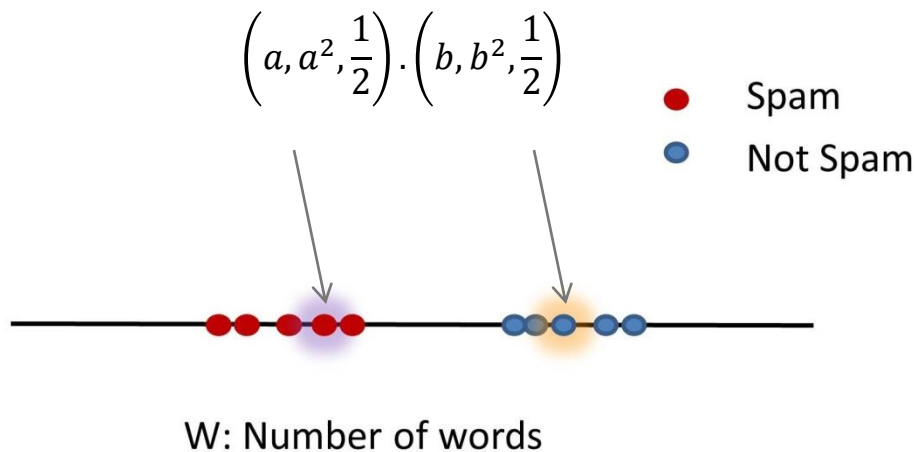
Consider $r = \frac{1}{2}$ $d = 2$

$$(a * b + \frac{1}{2})^2 = (a * b + \frac{1}{2})(a * b + \frac{1}{2}) = (a^2b^2 + ab + \frac{1}{4})$$

$$= (ab + a^2b^2 + \frac{1}{4})$$

$$= \left(a, a^2, \frac{1}{2}\right) \cdot \left(b, b^2, \frac{1}{2}\right)$$





$a=5$ and $b=10$

$$\left(5, 5^2, \frac{1}{2}\right) \cdot \left(10, 10^2, \frac{1}{2}\right) = (5 * 10) + (5^2 * 10^2) + \left(\frac{1}{2} * \frac{1}{2}\right) = 2550.25$$

- Use of **kernel** eliminates the need to actually transform the data from the low dimensions to high dimension
- Instead **kernel** use the dot products between each pair of points to compute the high dimensional relationships and find the optimal SVC

Radial Kernel

Radial Kernel

$$e^{-\gamma(a-b)^2}$$

γ : how much influence neighboring points have on classification

a and b are two observations or data points

A good value of γ can be found by using cross validation to determine which one is best

The main idea is that when we want to classify a new point, how the closest points from the training data are classified

Radial kernel is like polynomial kernel with $r=0$ and $d=\infty$

$$(a * b + r)^d = (a * b + 0)^\infty$$

More than one class

- One-vs-One (OvO) Approach
 - In this method, SVM trains multiple classifiers for every pair of classes.
 - If there are N classes, the number of classifiers trained is $\frac{N(N-1)}{2}$.
 - Each classifier differentiates between two classes at a time.
 - The final class for a new data point is determined using a majority voting scheme or a max-wins strategy.
 - Often more accurate
 - Computationally expensive

More than one class

- One-vs-Rest (OvR) Approach
 - In this method, N binary classifiers are trained, one for each class vs. all other classes combined.
 - During prediction, the classifier with the highest confidence score determines the final class.
 - Requires fewer classifiers N instead of $\frac{N(N-1)}{2}$.
 - Computationally more efficient than OvO
 - May be less accurate

REFERENCES

- *Many of the slide materials are from the following sources-*
- Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani. An Introduction to Statistical Learning : with Applications in R.
- Tan, Steinbach and Kumar's book on “An Introduction to Data Mining”

THANK YOU